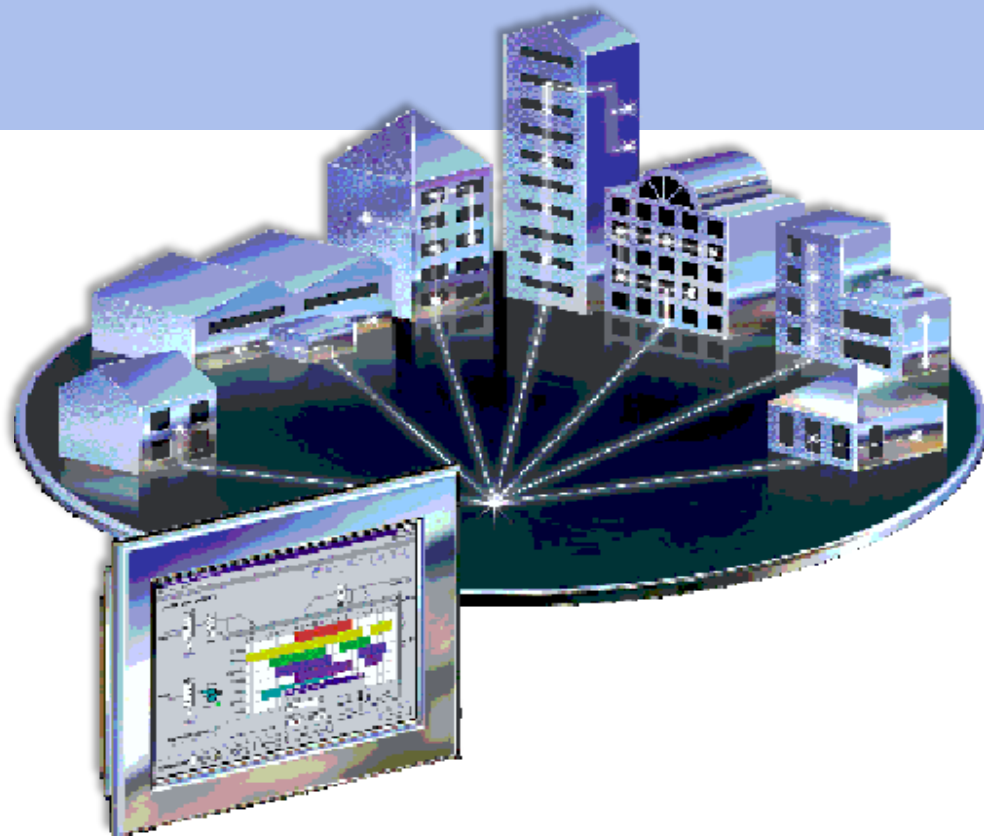


INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Práctica 2:

INTERCONEXIÓN CLIENTE-SERVIDOR



SISTEMAS DISTRIBUIDOS

Profesor || Chadwick Carreto Arellano

Grupo || 7CM1

Alumno || Areli Alejandra Guevara Badillo

Febrero 13, 2026

ANTECEDENTES

La interconexión cliente-servidor es un modelo de computación distribuida en el que los dispositivos de los usuarios (clientes) se conectan a un equipo centralizado (servidor) para acceder a recursos, aplicaciones, datos o servicios, estableciendo una relación de solicitud y respuesta.

El pilar de esta comunicación es el modelo TCP/IP (Transmission Control Protocol), el cual garantiza que los paquetes alcancen su destino (IP) de manera ordenada y sin pérdida de datos (TCP).

Para que las aplicaciones de software puedan interactuar con esta pila de protocolos, se utiliza una abstracción de programación conocida como "sockets" (conectores). Introducidos originalmente en 1983 con la versión 4.2BSD del sistema operativo Unix (conocidos como *Berkeley Sockets*), proporcionan una interfaz de programación de aplicaciones (API) estandarizada (POSIX).

Un socket representa un punto final (endpoint) en un enlace de comunicación bidireccional entre dos programas que se ejecutan en la red. En su implementación en lenguajes como Python, la API de sockets permite a los desarrolladores enlazar procesos a través de una dirección IP y un puerto específico, abstrayendo la complejidad del enrutamiento de hardware subyacente.

Existen dos tipos de sockets:

Stream Sockets (TCP): Orientados a conexión, fiables, garantizan el orden de llegada de paquetes.

Datagram Sockets (UDP): No orientados a conexión, más rápidos pero no garantizan la entrega ni el orden (ideales para streaming o juegos).

Componentes Fundamentales

- **Servidor:** Programa que espera pasivamente ("escucha") a que lleguen solicitudes en un puerto específico de una dirección IP.
- **Cliente:** Programa que inicia la conexión solicitando un servicio al servidor.
- **IP + Puerto:** Es la dirección única que define dónde se encuentra el servicio.

Proceso de Conexión (Modelo TCP Orientado a Conexión)

1. **Servidor crea el socket:** Se crea el socket y se vincula (bind) a una dirección IP y puerto.
2. **Servidor escucha (listen):** El servidor se pone en modo espera.
3. **Cliente crea el socket:** El cliente crea su propio socket y solicita la conexión al servidor.
4. **Aceptación (accept):** El servidor acepta la conexión, generando un nuevo socket para manejar ese cliente específico.
5. **Intercambio de datos:** Se utilizan funciones como send/read o recv/write para intercambiar información.
6. **Cierre:** Ambas partes cierran el socket para liberar recursos.

PLANTEAMIENTO DEL PROBLEMA

En la era actual de computación en la nube y servicios remotos, existe una tensión constante entre la accesibilidad de los datos y la privacidad del usuario. El objetivo principal de este proyecto es construir un sistema de Almacenamiento de Secretos con Conocimiento Cero (Zero-Knowledge Storage).

En el escenario planteado, un Cliente necesita guardar información sensible (contraseñas, claves privadas, notas confidenciales) en un Servidor remoto para asegurar su disponibilidad. Sin embargo, el requisito criptográfico fundamental es que el Servidor nunca debe poder leer la información que almacena. El servidor debe actuar simplemente como un depósito de datos cifrados ("caja ciega"), sin poseer las capacidades ni las claves para descifrar el contenido.

PROPUESTA DE SOLUCIÓN

Para abordar el problema, se propone una arquitectura Cliente-Servidor asíncrona basada en sockets TCP, bajo el principio de Privacidad Zero-Knowledge.

- **Nodo Cliente:** Responsable de la interfaz de usuario (UI), la gestión de claves maestras y las operaciones criptográficas. El cifrado ocurre *antes* de la transmisión de red.
- **Nodo Servidor:** Actúa como un repositorio ciego. Recibe blobs de datos cifrados, los asocia a un identificador único (`secret_id`) y los almacena. No posee las claves de descifrado.

Modelo de Seguridad

- **Cifrado Simétrico Autenticado:** Se utiliza el esquema Fernet (AES-128-CBC con HMAC) mediante la librería `cryptography`. Esto garantiza confidencialidad e integridad.
- **Gestión de Claves:** La clave de cifrado se genera y reside exclusivamente en el cliente. Nunca se transmite por la red.
- **Codificación:** Los datos cifrados (binarios) se codifican en Base64 para su transmisión segura dentro de paquetes JSON.

DESARROLLO E IMPLEMENTACIÓN

El desarrollo del sistema se llevó a cabo utilizando Python 3.14 como lenguaje principal, aprovechando su ecosistema robusto para operaciones de red, criptografía e interfaces gráficas. La implementación se dividió en dos módulos independientes pero complementarios: el Servidor Central y el Cliente Ligero con Interfaz Gráfica. Esta fase del proyecto se centra estrictamente en la comunicación 1 a 1 (un cliente, un servidor), estableciendo la lógica base antes de incorporar concurrencia múltiple. A continuación, se detalla la arquitectura técnica, las decisiones de diseño y la lógica interna de cada componente.

Entorno y Librerías

Para garantizar la seguridad y la funcionalidad de la conexión punto a punto, se seleccionaron las siguientes librerías estándar y de terceros:

- **socket**: Para la comunicación de bajo nivel mediante TCP/IP, estableciendo un canal fiable entre el cliente y el servidor.
- **json**: Para la serialización y deserialización de los paquetes de datos intercambiados.
- **cryptography.fernet**: Para implementar cifrado simétrico autenticado de alto nivel, asegurando que los datos sean ilegibles sin la clave correcta.
- **tkinter**: Para la construcción de la interfaz gráfica de usuario (GUI) nativa, permitiendo la interacción visual con el sistema.
- **base64**: Para la codificación de datos binarios cifrados en formato texto ASCII, asegurando una transmisión libre de errores a través del protocolo.
- **threading**: Utilizado exclusivamente en el lado del cliente para evitar el congelamiento de la interfaz gráfica durante las operaciones de red.

Arquitectura del Servidor (`server.py`)

El servidor está diseñado como un proceso persistente que escucha solicitudes entrantes en un puerto específico. Su implementación en esta fase se basa en un modelo de atención secuencial para una única conexión activa (ver Anexo A).

Configuración del Socket: Se inicializa un socket IPv4 de tipo stream (SOCK_STREAM). Se configura la opción SO_REUSEADDR para permitir la reutilización inmediata del puerto tras un reinicio.

```
1 server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
2 server.bind((HOST, PORT))
```

Bucle de Atención Principal: El servidor entra en un bucle de escucha (listen) y acepta una conexión (accept). Una vez establecida la conexión con el cliente, el servidor delega el manejo de esa sesión a una función dedicada (handle_client).

```
1 try:
2     server.bind((HOST, PORT))
3     server.listen()
4     print(f"[ESCUCHANDO] Servidor corriendo en {HOST}:{PORT}")
5
6     while True:
7         conn, addr = server.accept()
8         thread = threading.Thread(target=handle_client, args=(conn, addr))
9         thread.start()
10        print(f"[CONEXIONES ACTIVAS] {threading.active_count() - 1}")
```

Estructura de Almacenamiento: Se implementó un diccionario global en memoria `vault_storage = {}` que actúa como una tabla hash temporal. La clave es el `secret_id` (string) y el valor es el payload cifrado (string en Base64). Esta estructura permite búsquedas rápidas por identificador y es fácilmente sustituible por una base de datos persistente en futuras iteraciones.

```
1 # Estructura: { "id_secreto": "datos_cifrados_en_base64" }
2 vault_storage = {}
3
4 vault_storage[secret_id] = payload
5 response = {"status": "success", "message": "Secreto almacenado (Cifrado)"}
6 print(f"[ALMACENAMIENTO] ID: {secret_id} guardado por {addr}")
```

Manejo de Sesión: La función de manejo mantiene un bucle `while connected` para el cliente activo. Esto permite realizar múltiples operaciones (guardar, recuperar) dentro de una misma conexión TCP, reduciendo la sobrecarga de establecer handshakes repetidos para cada acción.

Arquitectura del Cliente (`cliente.py`)

El cliente combina la lógica de negocio con una interfaz gráfica interactiva, asegurando que la experiencia de usuario sea fluida, aunque la red tenga latencia (ver Anexo B).

Interfaz Gráfica (GUI): Se utilizó la librería `tkinter` organizando la ventana en marcos (`LabelFrame`) lógicos:

- **Conexión:** Botón para iniciar el socket y label de estado (conectado/desconectado).
- **Clave Maestra:** Campo para ingresar o generar la clave Fernet local.
- **Operaciones:** Campos para ID del secreto, Contenido en texto plano y botones de acción.
- **Feedback:** Label para mostrar resultados de operaciones o mensajes de error.

Responsividad de la UI: para evitar que la interfaz gráfica se congele mientras se espera una respuesta del servidor, las operaciones de red (`store_secret`, `retrieve_secret`) se ejecutan en un hilo secundario (`threading.Thread`). Esto permite al usuario mover la ventana o cancelar operaciones mientras se procesa la solicitud en segundo plano.

Gestión de Claves Locales: El cliente incluye una función para generar claves Fernet válidas (`Fernet.generate_key()`). Esta clave se muestra al usuario y debe ser gestionada por él. El sistema no almacena esta clave en disco ni la envía por la red, reforzando el modelo de seguridad donde el cliente es el único custodio de su acceso.

```
1 def generate_key(self):
2     key = Fernet.generate_key()
3     self.entry_key.delete(0, tk.END)
4     self.entry_key.insert(0, key.decode('utf-8'))
5     messagebox.showinfo("Clave Generada", "Guarda esta clave en un lugar seguro. Sin ella no podrás recuperar tus datos.")
```

Diseño del Protocolo de Comunicación

Dado que TCP es un protocolo orientado a flujos (stream) y no a mensajes, existe el riesgo de que los paquetes se fragmenten. Para garantizar la integridad de los mensajes JSON en esta conexión punto a punto, se implementó un Protocolo de Longitud Prefijada:

1. **Encabezado (Header):** Antes de enviar el contenido JSON, el emisor calcula la longitud en bytes del mensaje. Esta longitud se convierte a string y se rellena con espacios a la izquierda hasta ocupar exactamente 4 bytes.

```
1 msg_length = len(msg)
2 conn.send(str(msg_length).encode('utf-8').ljust(4))
```

2. **Cuerpo (Payload):** Después del encabezado, envían los bytes del mensaje JSON.
3. **Recepción:** El receptor primero lee exactamente 4 bytes, los decodifica para obtener la longitud esperada, y luego realiza una segunda lectura especificando exactamente esa cantidad de bytes. Esto asegura que el `json.loads()` reciba el objeto completo y válido, evitando errores de sintaxis por lecturas parciales.

Flujo Criptográfico Detallado

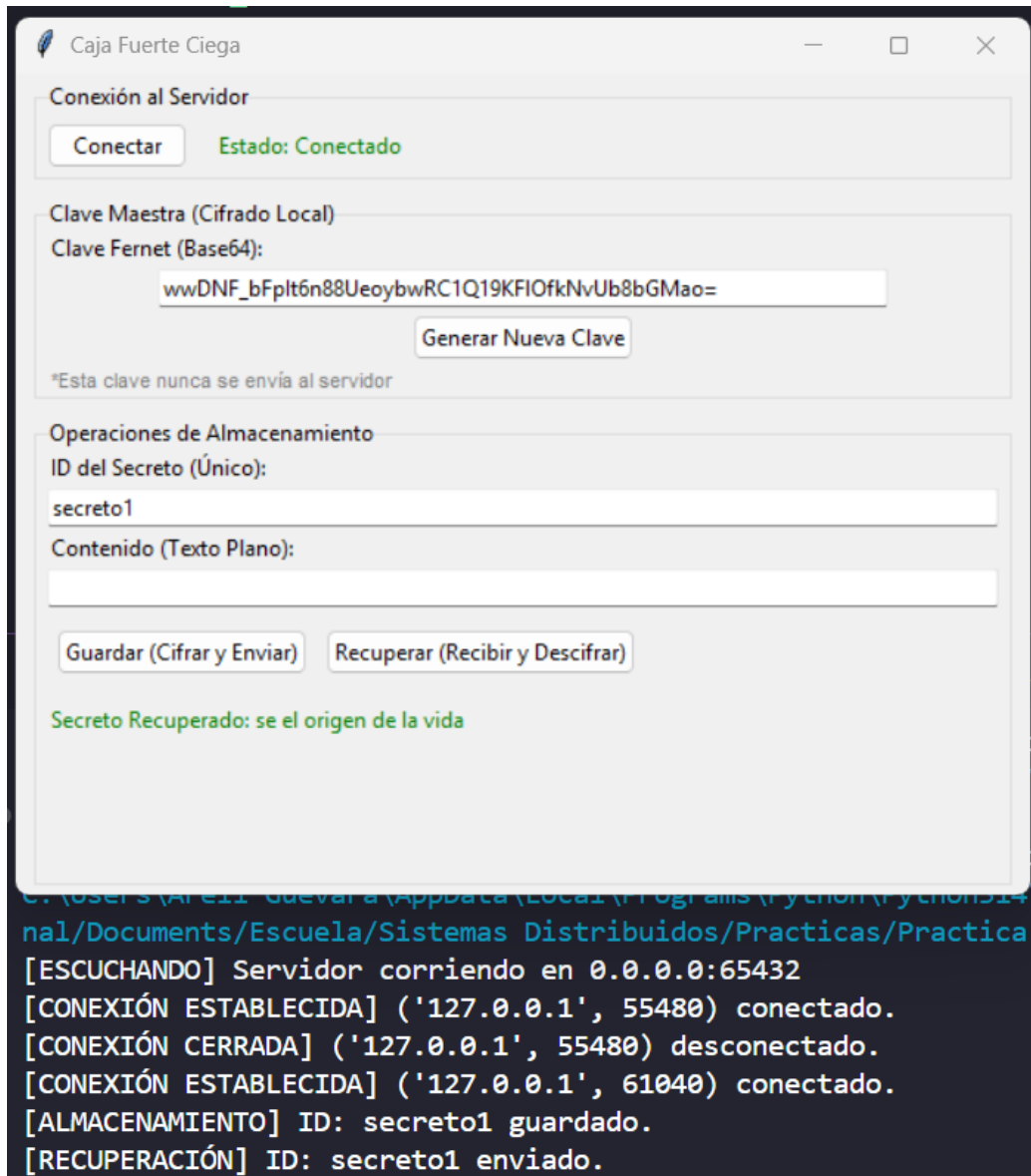
La seguridad del sistema reside en el proceso de cifrado/descifrado que ocurre exclusivamente en el cliente, manteniendo al servidor en ceguera respecto al contenido.

1. **Preparación (Cliente):**
 - El usuario ingresa texto plano (ej. "mi_secreto").
 - Se instancia la clase Fernet con la clave maestra proporcionada.
 - Se ejecuta `f.encrypt(texto.encode('utf-8'))`. Esto genera un token cifrado que incluye timestamp y firma HMAC para prevenir manipulaciones.
2. **Codificación para Transmisión:**
 - El token cifrado es binario. Para evitar problemas de codificación en JSON, se aplica `base64.b64encode()`.
 - El resultado es un string ASCII que se inserta en el payload JSON bajo la clave "payload".
3. **Almacenamiento (Servidor):**
 - El servidor recibe el string Base64 y lo guarda tal cual en su diccionario local. No tiene capacidad de decodificarlo ni descifrarlo.
4. **Recuperación y Descifrado (Cliente):**
 - El cliente recibe el string Base64 del servidor.
 - Aplica `base64.b64decode()` para recuperar los bytes originales del token Fernet.

- Ejecuta `f.decrypt(token)`. Si la clave es incorrecta o el token fue alterado, la librería lanza una excepción `InvalidToken`, la cual es capturada por el cliente para informar al usuario sin revelar información sensible.

RESULTADOS

Se verificó el establecimiento exitoso del canal de comunicación entre el cliente y el servidor. El sistema logró mantener una sesión estable que permitió realizar múltiples operaciones de guardado y recuperación de datos sin necesidad de reconectar en cada acción.



Los resultados confirman que la implementación actual cumple con los requisitos funcionales del sistema. El sistema opera de manera estable y segura, proporcionando una base sólida para la incorporación de concurrencia múltiple y persistencia de datos en futuras iteraciones.

CONCLUSIONES

Se implementó exitosamente un sistema de Almacenamiento de Secretos con Conocimiento Cero, lo cual permitió validar que es posible delegar el almacenamiento de datos sensibles a un servidor remoto sin comprometer la privacidad de la información, ya que la clave de cifrado reside exclusivamente en el cliente.

El uso del esquema Fernet para el cifrado simétrico y la codificación Base64 para la transmisión garantizó la confidencialidad e integridad de los datos durante el intercambio. Asimismo, la implementación del protocolo de longitud prefijada solucionó los problemas de fragmentación típicos en los flujos TCP, asegurando que los mensajes JSON se recibieran completos y sin errores de sintaxis.

Aunque la arquitectura actual opera bajo un modelo 1 a 1 de manera secuencial, el diseño modular del servidor y el cliente facilita la escalabilidad hacia un sistema distribuido con concurrencia múltiple en futuras iteraciones.

ANEXOS

A. Código del Servidor (servidor.py)

```
import socket
import json
import sys

# Configuración del Servidor
HOST = '0.0.0.0' # Escuchar en todas las interfaces disponibles
PORT = 65432     # Puerto estándar para el servicio

# Estructura: { "id_secreto": "datos_cifrados_en_base64" }
vault_storage = {}

def handle_client(conn, addr):
    """Maneja la comunicación con el cliente conectado."""
    print(f"[CONEXIÓN ESTABLECIDA] {addr} conectado.")
    connected = True

    while connected:
        try:
            # Recibir longitud del mensaje (Encabezado de 4 bytes)
            msg_length = conn.recv(4).decode('utf-8')
            # Si no se recibe longitud, el cliente se ha desconectado
            if not msg_length:
                break
```



```
msg_length = int(msg_length)

message = conn.recv(msg_length).decode('utf-8')
data = json.loads(message)

action = data.get("action")
secret_id = data.get("id")
payload = data.get("payload")

response = {"status": "error", "message": "Acción desconocida"}

# Procesar solicitudes
if action == "STORE": # Almacenar secreto
    if secret_id and payload:
        vault_storage[secret_id] = payload
        response = {"status": "success", "message": "Secreto almacenado (Cifrado)"}
        print(f"[ALMACENAMIENTO] ID: {secret_id} guardado.")
    else:
        response = {"status": "error", "message": "Faltan ID o Payload"}

elif action == "RETRIEVE": # Recuperar secreto
    if secret_id in vault_storage:
        response = {
            "status": "success",
            "message": "Secreto recuperado",
            "payload": vault_storage[secret_id]
        }
        print(f"[RECUPERACIÓN] ID: {secret_id} enviado.")
    else:
        response = {"status": "error", "message": "ID no encontrado"}

elif action == "DISCONNECT":
    connected = False
    response = {"status": "success", "message": "Desconectando"}

response_msg = json.dumps(response).encode('utf-8')
msg_length = len(response_msg)

# Longitud (4 bytes) + mensaje
conn.send(str(msg_length).encode('utf-8').ljust(4))
conn.send(response_msg)

except Exception as e:
    print(f"[ERROR] con {addr}: {e}")
    connected = False
```

Práctica 2: Interconexión Cliente-Servidor

```
conn.close()
print(f"[CONEXIÓN CERRADA] {addr} desconectado.")

def start_server():
    """Inicia el socket del servidor y acepta conexiones entrantes."""
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    try:
        server.bind((HOST, PORT))
        server.listen()
        print(f"[ESCUCHANDO] Servidor corriendo en {HOST}:{PORT}")

        while True:
            # Aceptar conexión
            conn, addr = server.accept()
            handle_client(conn, addr)

    except KeyboardInterrupt:
        print("\n[APAGADO] Servidor deteniéndose...")
        server.close()
        sys.exit()

if __name__ == "__main__":
    start_server()
```

B. Código del Cliente (cliente.py)

```
import socket
import threading
import json
import tkinter as tk
from tkinter import ttk, messagebox
from cryptography.fernet import Fernet
import base64

# Configuración del Cliente
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 65432

class CryptoVaultClient:
    def __init__(self, root):
        self.root = root
        self.root.title("Caja Fuerte Ciega")
        self.root.geometry("550x450")

        self.sock = None
        self.connected = False

        # --- Componentes de la UI ---
        self.create_widgets()
```

Práctica 2: Interconexión Cliente-Servidor

```
def create_widgets(self):
    # Frame de Conexión
    conn_frame = ttk.LabelFrame(self.root, text="Conexión al Servidor")
    conn_frame.pack(fill="x", padx=10, pady=5)

    ttk.Button(conn_frame, text="Conectar", command=self.connect_to_server).pack(side="left", padx=5, pady=5)
    self.lbl_status = ttk.Label(conn_frame, text="Estado: Desconectado", foreground="red")
    self.lbl_status.pack(side="left", padx=10)

    # Frame de Clave Criptográfica
    key_frame = ttk.LabelFrame(self.root, text="Clave Maestra (Cifrado Local)")
    key_frame.pack(fill="x", padx=10, pady=5)

    ttk.Label(key_frame, text="Clave Fernet (Base64):").pack(anchor="w", padx=5)
    self.entry_key = ttk.Entry(key_frame, width=65)
    self.entry_key.pack(padx=5, pady=2)

    ttk.Button(key_frame, text="Generar Nueva Clave", command=self.generate_key).pack(pady=2)
    ttk.Label(key_frame, text="*Esta clave nunca se envía al servidor", foreground="gray", font=("Arial", 8)).pack(anchor="w", padx=5)

    # Frame de Operaciones
    op_frame = ttk.LabelFrame(self.root, text="Operaciones de Almacenamiento")
    op_frame.pack(fill="both", expand=True, padx=10, pady=5)

    ttk.Label(op_frame, text="ID del Secreto (Único):").pack(anchor="w", padx=5)
    self.entry_id = ttk.Entry(op_frame)
    self.entry_id.pack(fill="x", padx=5, pady=2)

    ttk.Label(op_frame, text="Contenido (Texto Plano):").pack(anchor="w", padx=5)
    self.entry_content = ttk.Entry(op_frame)
    self.entry_content.pack(fill="x", padx=5, pady=2)

    btn_frame = ttk.Frame(op_frame)
    btn_frame.pack(fill="x", padx=5, pady=10)

    ttk.Button(btn_frame, text="Guardar (Cifrar y Enviar)", command=lambda: self.thread_action(self.store_secret)).pack(side="left", padx=5)
    ttk.Button(btn_frame, text="Recuperar (Recibir y Descifrar)", command=lambda: self.thread_action(self.retrieve_secret)).pack(side="left",
padx=5)

    self.lbl_result = ttk.Label(op_frame, text="Resultado: ", wraplength=500, foreground="blue", justify="left")
    self.lbl_result.pack(padx=5, pady=5, anchor="w")

def generate_key(self):
    """Genera una clave Fernet válida y la muestra en el campo."""
    key = Fernet.generate_key()
    self.entry_key.delete(0, tk.END)
    self.entry_key.insert(0, key.decode('utf-8'))
    messagebox.showinfo("Clave Generada", "Guarda esta clave en un lugar seguro. Sin ella no podrás recuperar tus datos.")

def connect_to_server(self):
    """Establece la conexión TCP con el servidor."""
    try:
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.connect((SERVER_HOST, SERVER_PORT))
        self.connected = True
        self.lbl_status.config(text="Estado: Conectado", foreground="green")
        self.lbl_result.config(text="Conexión exitosa. Listo para operar.", foreground="green")
    except Exception as e:
        messagebox.showerror("Error de Conexión", str(e))
        self.connected = False
        self.lbl_status.config(text="Estado: Error", foreground="red")
```

Práctica 2: Interconexión Cliente-Servidor

```
def send_request(self, data_dict):
    """Envía datos al servidor siguiendo el protocolo de longitud prefijada."""
    if not self.connected or not self.sock:
        raise Exception("No hay conexión con el servidor")

    msg = json.dumps(data_dict).encode('utf-8')
    msg_length = len(msg)

    self.sock.send(str(msg_length).encode('utf-8').ljust(4))
    self.sock.send(msg)

    length_msg = self.sock.recv(4).decode('utf-8')
    if not length_msg:
        raise Exception("Conexión cerrada por el servidor")

    length = int(length_msg)
    response = self.sock.recv(length).decode('utf-8')
    return json.loads(response)

def store_secret(self):
    """Lógica Criptográfica de Almacenamiento."""
    try:
        key = self.entry_key.get().encode('utf-8')
        if not key:
            raise ValueError("Se requiere una clave maestra")

        f = Fernet(key)

        content = self.entry_content.get().encode('utf-8')
        secret_id = self.entry_id.get()

        if not content or not secret_id:
            raise ValueError("ID y Contenido son obligatorios")

        encrypted_token = f.encrypt(content)
        payload_b64 = base64.b64encode(encrypted_token).decode('utf-8')

        # Envío al servidor
        response = self.send_request({
            "action": "STORE",
            "id": secret_id,
            "payload": payload_b64
        })

        self.lbl_result.config(text=f"Servidor dice: {response['message']}", foreground="green")
        if response['status'] == 'success':
            self.entry_content.delete(0, tk.END)

    except Exception as e:
        self.lbl_result.config(text=f"Error: {str(e)}", foreground="red")

def retrieve_secret(self):
    """Lógica Criptográfica de Recuperación."""
    try:
        key = self.entry_key.get().encode('utf-8')
        if not key:
            raise ValueError("Se requiere una clave maestra")

        f = Fernet(key)

        secret_id = self.entry_id.get()
        if not secret_id:
```

Práctica 2: Interconexión Cliente-Servidor

```
        raise ValueError("ID es obligatorio")

    # Solicitud al servidor
    response = self.send_request({
        "action": "RETRIEVE",
        "id": secret_id
    })

    if response['status'] == 'success':
        # Decodificar y Descifrar
        payload_b64 = response['payload']
        encrypted_token = base64.b64decode(payload_b64)
        decrypted_content = f.decrypt(encrypted_token).decode('utf-8')

        self.lbl_result.config(text=f"Secreto Recuperado: {decrypted_content}", foreground="green")
    else:
        self.lbl_result.config(text=f"Error: {response['message']}", foreground="red")

    except Exception as e:
        self.lbl_result.config(text=f"Error (Clave incorrecta o red): {str(e)}", foreground="red")

def thread_action(self, target_func):
    if not self.connected:
        messagebox.showwarning("Atención", "Debes conectarte al servidor primero.")
        return

    thread = threading.Thread(target=target_func)
    thread.start()

if __name__ == "__main__":
    root = tk.Tk()
    app = CryptoVaultClient(root)
    root.mainloop()
```